IntoData Knowledge Base Project

Implementation Plan & Roadmap

Project Duration: February 3 - June 30, 2026
Intern-Led Initiative
Target Users: All departments (Developers, HR, Sales, PM, Support, Finance, Legal, Marketing, Leadership)

Executive Summary

This document outlines the implementation strategy for building IntoData's centralized knowledge base system. The system will enable natural language querying across all company documentation, code repositories, communications, and historical project data. The roadmap follows a phased approach prioritized using MoSCoW methodology, ensuring delivery of core functionality within the internship timeline while establishing a foundation for future enhancements.

---

1. Project Scope Definition (MoSCoW Prioritization)

MUST HAVE - Phase 1 & 2 (February - April)

Core MVP delivering immediate business value

1.1 Unified Search Interface

Objective: Provide a single entry point for knowledge retrieval across the organization.

Technical Requirements:

- Web-based user interface supporting natural language queries
- Technology stack: Streamlit (Python-based) for rapid prototyping and deployment
- Query input validation and preprocessing
- Response display with source attribution
- Basic query history tracking per session

Success Criteria:

- Users can submit queries in plain English
- System responds within 5 seconds for standard queries
- Interface accessible via internal network URL
- Mobile-responsive design for field staff access

Dependencies:

- Python 3.9+ environment
- Internal hosting infrastructure or cloud deployment approval

---

1.2 Document Ingestion Pipeline

Objective: Automate the extraction and processing of structured company documents.

Data Sources - Priority Order:

1. **Sales & Business Development Documents** (Immediate ROI)
    - Proposals and quotations (PDF, DOCX)
    - Client presentations (PPTX)
    - Contract templates and standard terms
    - Case studies and success stories
2. **HR & Operations Documents**
    - Employee handbooks and policies
    - Job description templates
    - Onboarding documentation
    - Training materials and certifications
3. **Technical Documentation**
    - System integration guides
    - Architecture diagrams (extract text descriptions)
    - API documentation
    - Deployment runbooks

Technical Implementation:

- Connector development for SharePoint/Google Drive APIs
- Support for file formats: PDF, DOCX, PPTX, XLSX (metadata only)
- Automated polling mechanism (initially: daily batch processing)
- Document metadata extraction (author, date, department, tags)
- Chunking strategy: 500-token segments with 50-token overlap for context preservation

Processing Pipeline:

Source System → API Extraction → Format Parsing → Text Chunking →
Metadata Tagging → Embedding Generation → Vector Storage

Success Criteria:

- Pipeline processes 50+ documents in initial test set without errors
- Successfully extracts text from all supported formats
- Maintains document lineage and metadata throughout pipeline
- Error logging and notification system for failed ingestions

Dependencies:

- API credentials for SharePoint/Google Drive
- Read-only access permissions to target document libraries
- Initial document inventory and categorization

1.3 Vector Database Infrastructure

Objective: Establish the core storage and retrieval engine for semantic search.

Technology Selection: Qdrant

- **Rationale:** Open-source, Docker-deployable, high performance, strong Python SDK support
- **Alternative considered:** Pinecone (ruled out due to cost for MVP phase)

Architecture:

- Initial deployment: Docker container on dedicated VM
- Storage: SSD-backed persistent volumes
- Vector dimensions: 1536 (OpenAI text-embedding-3-small)
- Initial capacity planning: 100,000 document chunks (scalable to 1M+)

Configuration:

- Collection per data source type (documents, communications, code)
- Indexing: HNSW (Hierarchical Navigable Small World) algorithm
- Distance metric: Cosine similarity
- Backup strategy: Daily snapshots to network storage

Success Criteria:

- Vector database operational with 99%+ uptime
- Query response time under 200ms for single queries
- Successfully stores and retrieves embeddings for test dataset
- Backup and restore procedures documented and tested

Dependencies:

- Infrastructure approval for VM deployment
- Docker environment setup
- Network access and firewall rules configuration

1.4 RAG (Retrieval-Augmented Generation) Pipeline

Objective: Implement the core intelligence layer that retrieves relevant information and synthesizes answers.

Technical Architecture:

Retrieval Component:

- Query embedding using text-embedding-3-small
- Top-K retrieval (K=5 initially, tunable)
- Relevance scoring threshold: 0.7+ cosine similarity
- Context window management: Maximum 3000 tokens for LLM

Generation Component:

- Primary LLM: OpenAI GPT-4o via API
- Fallback: Claude 3.5 Sonnet (cost optimization for routine queries)
- Temperature: 0.1 (prioritize accuracy over creativity)
- Max response tokens: 500 (concise answers)

System Prompt Architecture:

```
Role: You are IntoData's knowledge assistant
Context: [Retrieved document chunks with metadata]
Guidelines:
- Answer only from provided context
- Cite specific documents and page numbers
- If information is not in context, state "I don't have that information"
- For ambiguous queries, ask clarifying questions
- Flag outdated information (documents older than 2 years)
```

Citation System:

- Inline citations: [Source: filename, Page: X]
- Confidence scoring for each retrieved chunk
- Expandable source preview in UI

Success Criteria:

- System correctly answers 80%+ of test queries about documented processes
- Zero hallucinations (verified through blind testing)
- All answers include source citations
- Response relevance rated 4+/5 by test users

Dependencies:

- OpenAI API key with sufficient quota
- Access to Claude API (optional, for fallback)
- Test query dataset prepared from common use cases

1.5 Ephemeral Communications Ingestion

Objective: Capture tribal knowledge from Slack/Teams conversations to prevent knowledge loss.

Target Channels - Priority Order:

1. #dev-help, #technical-support (developer Q&A)
2. #sales-support, #proposals (sales knowledge)
3. #general-announcements (company updates)
4. #project-postmortems (lessons learned)

Technical Implementation:

- Slack API integration using Bolt SDK (Python)
- Historical message export (initial backfill: 12 months)
- Thread context preservation (parent-child message relationships)
- Real-time ingestion for ongoing messages (post-MVP)

Data Processing:

- Filter out noise: reactions, short messages (<20 words), bot messages
- Metadata capture: channel, author, timestamp, thread context
- Privacy considerations: exclude private channels and DMs
- Retention policy: 2-year rolling window

Search Optimization:

- Tag messages by topic using keyword extraction
- Identify "solution messages" (replies marked as helpful, reactions with checkmarks)

- Link related threads using embedding similarity

Success Criteria:

- Successfully ingest 6 months of Slack history from 5 priority channels
- Users can retrieve relevant past conversations with queries like "How did we solve X?"
- Thread context properly maintained (can see full conversation, not just snippets)
- Response time under 3 seconds for Slack-based queries

Dependencies:

- Slack workspace admin approval
- API token with message history access
- Legal/HR review of data privacy implications

SHOULD HAVE - Phase 3 (May)

Security, trust, and usability enhancements

2.1 Role-Based Access Control (RBAC) Gateway

Objective: Ensure users only access information appropriate to their role and department.

Security Model:

User Groups:

- Public (all employees)
- HR_Sensitive (HR officers, executives)
- Finance_Sensitive (finance team, executives)
- Sales_Confidential (sales team, executives)
- Engineering (developers, technical leads)
- Executive (C-level, full access)

Document Classification:

- Auto-tagging based on source location and metadata
- Manual classification for sensitive documents
- Inheritance model: files in "HR Policies" folder automatically tagged HR_Sensitive

Implementation:

- PostgreSQL database for user-role mappings

- LDAP/Active Directory integration for user authentication
- Vector database query filtering: `filter={"access_groups": user.groups}`
- Audit logging for all queries and document access

Access Control Flow:

```
User Query → Authentication → Role Lookup →
Query Filtering by Access Groups → Vector Search →
Results Filtering → Response Generation
```

Success Criteria:

- Zero false positives (users never see unauthorized content)
- Access control tested with 100+ query scenarios across all user groups
- Audit log captures all access attempts
- Permission denied messages are user-friendly and don't leak information

Dependencies:

- HR provides employee-role mappings
- Document owners classify sensitivity levels
- IT provides LDAP integration support

2.2 Source Citations & Trust Indicators

Objective: Build user confidence through transparent source attribution.

Citation Features:

Document-Level:

- Filename and document title
- Author and last modified date
- Department/owner
- Direct link to source document

Chunk-Level:

- Page number or section heading
- Surrounding context preview
- Relevance confidence score (0-100%)

UI Components:

- Inline citations within answer text
- Expandable "Sources" panel showing all referenced documents
- Visual indicators for document age (e.g., documents over 1 year flagged as "potentially outdated")
- "Was this helpful?" feedback mechanism

Trust Signals:

- Highlight official company policies vs. informal communications
- Show document approval status if available
- Display review dates for critical documents

Success Criteria:

- 100% of answers include at least one source citation
- Users can navigate to original document in ≤2 clicks
- User trust rating: 4.5+/5 in post-deployment survey
- Source preview loads within 1 second

Dependencies:

- Document metadata enrichment
- UI/UX design review
- User feedback collection mechanism

2.3 Metadata Filtering & Advanced Search

Objective: Enable users to narrow search scope for more precise results.

Filter Dimensions:

Temporal:

- Date range selector (last week, month, quarter, year, custom)
- "Show only recent documents" toggle (less than 6 months old)

Categorical:

- Department filter (Sales, HR, Engineering, Finance, etc.)
- Document type (proposal, policy, technical doc, email)
- Project/client name

Quality:

- Official vs. informal content
- Approved/reviewed status
- Author credibility ranking

Implementation:

- Filter UI: Multi-select dropdowns and date pickers
- Backend: Metadata-based pre-filtering before vector search
- Query optimization: Reduce search space before semantic matching

Search Modes:

- **Broad search:** Default, searches across all accessible content
- **Focused search:** User-defined filters applied
- **Similar documents:** Find documents similar to a selected result

Success Criteria:

- Filters reduce irrelevant results by 60%+
- Filter application doesn't increase query time by more than 500ms
- Users actively use filters in 40%+ of queries (analytics tracking)

Dependencies:

- Comprehensive metadata extraction during ingestion
- Analytics infrastructure for usage tracking

COULD HAVE - Phase 4 (June)

Advanced capabilities if time permits

3.1 Legacy Code Ingestion & "Translation Layer"

Objective: Make internal code repositories searchable through natural language, enabling code reuse and reducing duplicate development.

Challenge: Raw code is difficult to search semantically. Solution: Create human-readable descriptions of code functions and classes, then vectorize the descriptions while storing original code as payload.

Implementation Strategy:

Repository Selection (MVP):

- Focus on 1-2 high-value repositories initially
- Priority: Common utility libraries and frequently referenced ETL scripts

- Target: 50-100 reusable functions

Translation Process:

```
Code Repository → Function Extraction →
LLM-Generated Description → Embedding →
Vector Storage (with original code in payload)
```

LLM Prompt Template:

```
Analyze this Python function and provide:
1. A one-sentence description of what it does
2. Input parameters and their types
3. Return value and type
4. Common use cases
5. Any dependencies or requirements

Function:
[code here]
```

Pseudo-Implementation:

```python
```

```
def ingest_code_repository(repo_path, language="python"):
    for file in get_files(repo_path, extension=language):
        functions = extract_functions(file)

        for func in functions:
            # Generate natural language description
            description = llm.generate(
                prompt=f"Describe this {language} function: {func.code}",
                model="gpt-4o"
            )

            # Create searchable metadata
            metadata = {
                "type": "code",
                "language": language,
                "file_path": file.path,
                "function_name": func.name,
                "description": description,
                "original_code": func.code
            }

            # Vectorize the description, store code as payload
            vector_db.upsert(
                vector=embed(description),
                payload=metadata
            )
```

Query Examples:

- "Show me functions for handling CSV file uploads" → Returns relevant ETL utilities
- "How do we connect to Snowflake in Python?" → Returns connection helper functions
- "Find all code that reverses a list" → Returns list manipulation utilities

Success Criteria:

- Successfully ingest and describe 50+ functions
- Developers can find relevant code using natural language in 70%+ of test queries
- Code descriptions are accurate (verified by senior developers)
- System reduces "reinventing the wheel" development time by measurable amount

Risks & Mitigation:

- **Risk:** LLM descriptions may be inaccurate for complex functions
    - **Mitigation:** Manual review of generated descriptions for critical code

- **Risk:** High API costs for large codebases
  - **Mitigation:** Batch processing, caching, incremental updates only
- **Risk:** Outdated code remains in system
  - **Mitigation:** Re-process code when git commits detected, flag old code versions

---

3.2 Visual Dependency Maps & Knowledge Graphs

Objective: Provide visual representations of how documents, projects, and contracts relate to each other.

Use Cases:

- Show all documents related to a specific client
- Map project dependencies and deliverables
- Visualize contract relationships (master agreements, amendments, SOWs)
- Display team knowledge networks (who knows what)

Technical Approach:

- Graph database: Neo4j (document nodes, relationship edges)
- Relationships extracted from:
  - Explicit links (hyperlinks in documents)
  - Metadata (project tags, client names)
  - Semantic similarity (documents about the same topic)
  - Temporal relationships (documents created in the same project phase)

Visualization:

- Interactive graph UI using D3.js or Cytoscape.js
- Node types: Documents, People, Projects, Clients
- Edge types: References, Related_To, Authored_By, Part_Of

Success Criteria:

- Visual graph renders within 3 seconds for typical queries
- Users can navigate related documents visually
- Graph provides insights not obvious from text search alone

3.3 Slack Bot Integration

Objective: Enable knowledge access directly within Slack without requiring users to switch to web UI.

Features:

- Mention bot: `@IntoDataKB what's our vacation policy?`
- Slash command: `/ask what's our vacation policy?`
- Interactive response with source links
- "Ask a follow-up" buttons for conversational queries

Implementation:

- Slack Bot using Bolt SDK
- Webhook integration with existing RAG pipeline
- Rate limiting: 10 queries per user per hour (prevent abuse)
- Private response option for sensitive queries

Success Criteria:

- Bot responds within 5 seconds
- Users prefer Slack interface for quick queries (analytics)
- Bot adoption rate: 50%+ of employees use it monthly

WON'T HAVE (Explicitly Out of Scope)

Capabilities deferred to future phases or permanently excluded

Write-Back Capabilities

- **What:** System will not edit documents, commit code, or modify source data
- **Rationale:** Read-only access reduces risk, simplifies permissions, prevents data corruption
- **Alternative:** System can suggest edits, which users manually implement

Real-Time Database Replication

- **What:** No sub-second syncing of data sources
- **Rationale:** Batch processing (daily/nightly) sufficient for knowledge base use case, reduces infrastructure complexity
- **Schedule:** Daily ingestion runs at 2 AM local time

Multi-Language Support (Initial Release)

- **What:** UI and responses in English only
- **Rationale:** Company operates primarily in English, localization adds complexity

- **Future:** Can be added post-MVP if international expansion occurs

Video/Audio Content Ingestion

- **What:** No transcription or analysis of recorded meetings, training videos
- **Rationale:** Requires additional transcription infrastructure, lower ROI than documents
- **Future:** Could leverage Whisper API in Phase 2 if demand exists

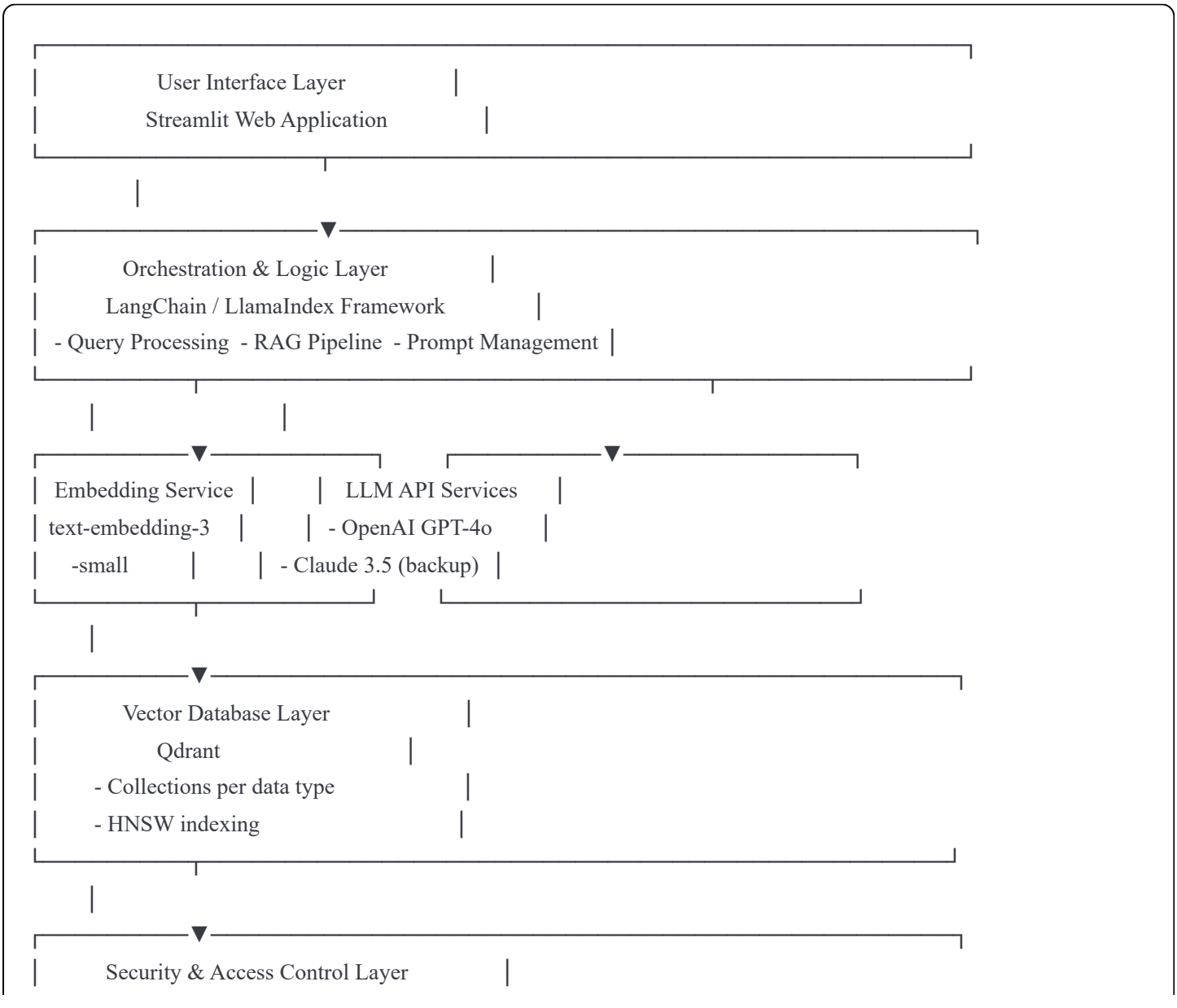
Automated Document Classification

- **What:** No ML-based automatic sensitivity tagging
- **Rationale:** Security-critical, requires manual review and validation
- **Approach:** Manual classification during initial ingestion, rule-based automation later

---

2. Technical Stack Specification

Architecture Overview





Technology Selection Matrix

Layer	Technology	Justification	Alternatives Considered
Frontend	Streamlit	Rapid Python-based prototyping, minimal frontend code, excellent for data apps	React (too complex for timeline), Gradio (less flexible)
Orchestration	LangChain	Industry standard, extensive documentation, active community	LlamaIndex (considered equivalent, chose LangChain for team familiarity)
Vector DB	Qdrant	Open-source, Docker-deployable, high performance, excellent Python SDK	Pinecone (expensive), Weaviate (less mature), Chroma (local-only)
Embeddings	text-embedding-3-small	Cost-effective ($0.02/1M tokens), 1536 dimensions, high quality	text-embedding-3-large (3x cost, minimal accuracy gain for this use case)
LLM	OpenAI GPT-4o	Superior reasoning, excellent instruction following, reliable API	Claude 3.5 Sonnet (used as fallback, slightly slower), GPT-3.5-Turbo (insufficient reasoning)
RBAC Database	PostgreSQL	Reliable, well-known, ACID compliant, excellent Python libraries	MongoDB (less suited for relational auth data), SQLite (not production-grade)
Authentication	LDAP/Active Directory	Enterprise standard, existing infrastructure integration	Custom auth (reinventing wheel), OAuth (overcomplicated for internal app)

Development Environment

Required Software:

- Python 3.9+ (3.11 recommended for performance)
- Docker Desktop (for Qdrant deployment)
- Git (version control)
- Visual Studio Code or PyCharm (recommended IDEs)

Python Dependencies:

```
langchain>=0.1.0
openai>=1.0.0
qdrant-client>=1.7.0
streamlit>=1.30.0
pypdf2>=3.0.0
python-docx>=1.0.0
python-pptx>=0.6.0
slack-sdk>=3.23.0
psycopg2-binary>=2.9.0
python-dotenv>=1.0.0
```

Infrastructure Requirements

Development Environment:

- Local development: Developer laptops (minimum 16GB RAM)
- Version control: Git repository (GitHub/GitLab)
- Package management: pip with requirements.txt

Production Environment:

- **Compute:**
  - Application server: 4 vCPU, 16GB RAM (Streamlit + orchestration)
  - Vector database: 8 vCPU, 32GB RAM, 500GB SSD (Qdrant)
- **Storage:**
  - Document storage: 1TB network-attached storage
  - Database: 100GB SSD (PostgreSQL)
  - Backups: 2TB capacity for historical snapshots
- **Network:**
  - Internal network access only (no public internet exposure)
  - Firewall rules: Allow HTTPS (443) from corporate network
  - VPN access for remote employees

Estimated Costs (Cloud Deployment):

- Compute: $300-500/month
- Storage: $100-150/month
- API costs (OpenAI): $200-400/month (estimated based on 10,000 queries/month)
- **Total:** ~$600-1,050/month operational cost

---

3. Implementation Roadmap

Phase 1: Foundation & Data Ingestion

Timeline: February 3 - March 15, 2026 (6 weeks)
Objective: Establish core infrastructure and ingest initial test dataset

Week 1-2: Environment Setup & Planning (Feb 3-16)

Deliverables:

- Development environment configured on intern laptop
- Git repository created with project structure
- API keys obtained (OpenAI, Qdrant Cloud trial)
- Initial document inventory completed

Tasks:

1. Install Python 3.11, Docker Desktop, IDE
2. Set up virtual environment and install dependencies
3. Create project structure:

```
intodata-kb/
├── src/
│   ├── ingestion/
│   ├── retrieval/
│   ├── generation/
│   └── ui/
├── data/
│   ├── raw/
│   └── processed/
├── config/
├── tests/
└── docs/
```

4. Obtain necessary credentials and access permissions
5. Conduct kickoff meeting with guidance counselor

Success Metrics:

- Development environment passes health check script
- Team can commit and push code to repository
- All team members have necessary access credentials

Risk: API key approval delays
Mitigation: Request keys in first 2 days, use free tiers/trials if needed

---

Week 3-4: Data Pipeline Development (Feb 17 - March 2)

Deliverables:

- Working document ingestion pipeline
- 50 high-value documents processed into vector database
- Pipeline documentation and runbook

Tasks:

1. **Document Collection:**
    - Work with Sales to export 20 recent proposals (PDF)
    - Work with HR to export 15 policy documents (DOCX)
    - Collect 15 technical integration guides (PDF/DOCX)
2. **Pipeline Development:**
    - Implement PDF loader using PyPDF2
    - Implement DOCX loader using python-docx
    - Implement PPTX loader using python-pptx
    - Build chunking function (500 tokens, 50 overlap)
    - Integrate OpenAI embeddings API
    - Connect to Qdrant and create collections
3. **Testing:**
    - Process test documents through pipeline
    - Verify chunk quality and overlap
    - Validate embeddings stored correctly
    - Test error handling for malformed files

Code Milestone:

```
python
```

```python
# ingestion_pipeline.py - Simplified example
from langchain.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from qdrant_client import QdrantClient

def ingest_document(file_path, metadata):
    # Load document
    loader = PyPDFLoader(file_path)
    documents = loader.load()

    # Chunk text
    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=500,
        chunk_overlap=50
    )
    chunks = text_splitter.split_documents(documents)

    # Generate embeddings
    embeddings_model = OpenAIEmbeddings(model="text-embedding-3-small")

    # Store in Qdrant
    client = QdrantClient(url="http://localhost:6333")
    for i, chunk in enumerate(chunks):
        vector = embeddings_model.embed_query(chunk.page_content)
        client.upsert(
            collection_name="documents",
            points=[{
                "id": f"{metadata['doc_id']}_{i}",
                "vector": vector,
                "payload": {
                    "text": chunk.page_content,
                    "source": metadata['filename'],
                    "page": chunk.metadata.get('page', 0),
                    "department": metadata['department']
                }
            }]
        )
```

Success Metrics:

- Pipeline processes 50 documents without errors
- Average processing time: <30 seconds per document
- Embedding quality verified through spot checks
- Error logs capture and report all failures

Risk: Document format inconsistencies
Mitigation: Build robust error handling, maintain "problematic files" log for manual review

---

Week 5-6: Basic Retrieval System (March 3-15)

Deliverables:

- Command-line query tool functional
- RAG pipeline retrieving relevant chunks
- First successful end-to-end query demonstration

Tasks:

1. **Retrieval Implementation:**
    - Build query function that embeds user question
    - Implement vector similarity search (top-5 results)
    - Set relevance threshold (0.7 cosine similarity)
    - Extract and format retrieved chunks
2. **Generation Integration:**
    - Connect to OpenAI GPT-4o API
    - Design system prompt for knowledge assistant
    - Implement context window management
    - Add citation formatting
3. **Testing & Validation:**
    - Create test query set (20 questions from use cases)
    - Run queries and evaluate answer quality
    - Measure response times
    - Iterate on prompts to reduce hallucination

Test Queries:

- "What is IntoData's vacation policy?"
- "Show me the pricing structure for data migration projects"
- "What's the standard onboarding process for new developers?"
- "How do we handle GDPR compliance for EU clients?"

Success Metrics:

- 80%+ of test queries return relevant information

- Zero complete hallucinations (making up information)
- All answers include source citations
- Average end-to-end response time: <5 seconds

Risk: Retrieval returns irrelevant chunks
Mitigation: Tune similarity thresholds, experiment with chunk sizes, add metadata filtering

---

Phase 2: User Interface & Communications (March 16 - April 30)

Timeline: 7 weeks
Objective: Make system accessible to non-technical users, integrate Slack history

Week 7-9: Web Application Development (March 16 - April 6)

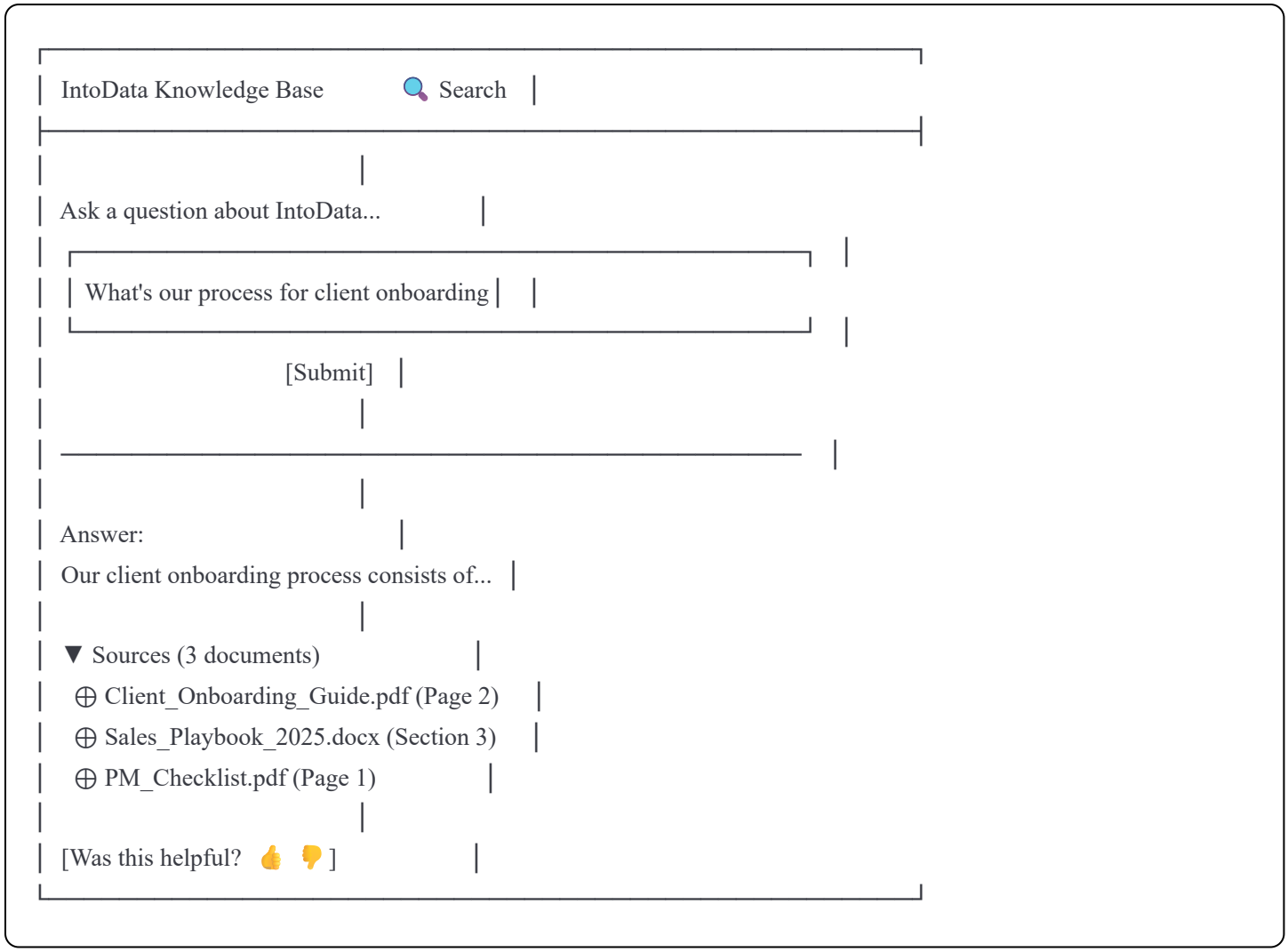
Deliverables:

- Streamlit web application deployed internally
- User authentication integrated
- Basic analytics dashboard

Tasks:

1. **UI Development:**
    - Build query input interface
    - Display formatted answers with sources
    - Add "Sources" expandable section
    - Implement query history per session
    - Create simple admin dashboard (query volume, common queries)
2. **Deployment:**
    - Containerize application with Docker
    - Deploy to internal VM/server
    - Configure reverse proxy (nginx)
    - Set up HTTPS with internal certificate
    - Create monitoring alerts (uptime, error rates)
3. **User Testing:**
    - Recruit 5 pilot users (1 per department)
    - Conduct usability sessions
    - Collect feedback on interface and results
    - Iterate based on feedback

UI Mockup Structure:



Success Metrics:

- Application uptime: 99%+ during business hours
- User satisfaction: 4/5 average rating
- 90%+ of pilot users successfully complete queries independently
- Page load time: <2 seconds

Risk: Server deployment delays
Mitigation: Use Streamlit Cloud as temporary hosting if internal deployment blocked

---

Week 10-12: Slack Integration (April 7-27)

Deliverables:

- Slack message history ingested (6+ months)
- Conversation threads properly indexed
- Slack queries return relevant historical discussions

Tasks:

1. **Slack API Setup:**
    - Create Slack app in workspace
    - Request admin approval for message export
    - Configure OAuth scopes (channels:history, channels:read)
2. **Data Export:**
    - Export message history from 5 priority channels
    - Parse JSON response format
    - Preserve thread relationships (parent-child)
    - Filter out noise (reactions, short messages, bots)
3. **Message Processing:**
    - Chunk conversations by thread
    - Extract metadata (channel, author, timestamp, reactions)
    - Identify "solution messages" (high reaction count, marked helpful)
    - Generate embeddings per message/thread
4. **Integration Testing:**
    - Query historical technical issues: "How did we solve the pagination bug?"
    - Verify thread context appears in results
    - Ensure recent messages appear in search

Slack-Specific Considerations:

- Privacy: Only public channels, no DMs or private channels
- Retention: 2-year rolling window (delete older messages)
- Rate limiting: Batch processing to avoid API throttling
- User mentions: Anonymize or link to employee directory

Success Metrics:

- 6 months of Slack history successfully ingested
- Users can find past solutions to recurring problems
- Thread context properly maintained (see full conversation, not isolated messages)
- Slack-based queries respond as quickly as document queries

Risk: Slack admin approval delays
Mitigation: Prepare privacy/security justification document early, escalate to leadership if needed

---

Week 13: Phase 2 Testing & Refinement (April 28-30)

Deliverables:

- Comprehensive testing report
- Bug fixes and performance improvements
- Updated documentation

Tasks:

- Conduct load testing (simulate 50 concurrent users)
- Fix identified bugs and performance bottlenecks
- Update user documentation
- Prepare demo for stakeholder review

Phase 3: Security & Production Readiness (May 1-31)

Timeline: 4 weeks
Objective: Implement access controls, optimize performance, prepare for company-wide rollout

Week 14-15: RBAC Implementation (May 1-15)

Deliverables:

- PostgreSQL database with user-role mappings
- Document access controls enforced
- Audit logging operational

Tasks:

1. **Database Setup:**
    - Design schema (users, roles, permissions, audit_log tables)
    - Import employee data from HR system
    - Map employees to role groups
2. **Document Classification:**
    - Tag existing documents with access groups
    - Create classification guidelines for future documents
    - Implement automated tagging rules (folder-based)
3. **Query Filtering:**
    - Modify retrieval logic to filter by user permissions
    - Add permission checks before displaying results
    - Implement "access denied" graceful messaging

4. **Security Testing:**
    - Create test users with different permission levels
    - Verify users cannot access unauthorized content
    - Test boundary cases (user with no groups, user with multiple groups)
    - Penetration testing (attempt to bypass filters)

Schema Example:

```sql
CREATE TABLE users (
    user_id SERIAL PRIMARY KEY,
    email VARCHAR(255) UNIQUE,
    name VARCHAR(255),
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE roles (
    role_id SERIAL PRIMARY KEY,
    role_name VARCHAR(100) UNIQUE
);

CREATE TABLE user_roles (
    user_id INT REFERENCES users(user_id),
    role_id INT REFERENCES roles(role_id),
    PRIMARY KEY (user_id, role_id)
);

CREATE TABLE audit_log (
    log_id SERIAL PRIMARY KEY,
    user_id INT REFERENCES users(user_id),
    query_text TEXT,
    timestamp TIMESTAMP DEFAULT NOW(),
    results_count INT,
    accessed_documents TEXT[]
);
```

Success Metrics:

- 100% of test cases pass (users only see authorized content)
- Zero false positives in permission testing
- Audit log captures all queries with user attribution
- Average query time increase: <300ms due to filtering

Risk: Complex permission matrix
Mitigation: Start with simple groups, iterate to more granular permissions in future phases

---

Week 16: System Optimization (May 16-23)

Deliverables:

- Performance benchmarks documented
- Optimized query paths
- System monitoring dashboard

Tasks:

1. **Performance Profiling:**
    - Identify slow queries and bottlenecks
    - Optimize vector search parameters (adjust top-K, similarity threshold)
    - Implement caching for frequent queries
    - Database query optimization (add indexes)
2. **Prompt Engineering:**
    - Refine system prompts to reduce hallucination
    - Add explicit instructions: "If not in context, say 'I don't know'"
    - Test answer quality with refined prompts
    - Create prompt version control
3. **Monitoring Setup:**
    - Implement application logging (errors, warnings, info)
    - Set up metrics dashboard (queries/day, response times, error rates)
    - Configure alerts (system down, high error rate, slow queries)
4. **Documentation:**
    - User manual (how to use the system)
    - Administrator guide (how to maintain and troubleshoot)
    - API documentation (for future integrations)

Success Metrics:

- 95th percentile query response time: <3 seconds
- Cache hit rate: >30% (common queries)
- System availability: 99.5%+
- Documentation completeness: all features covered

Week 17: Pre-Launch Validation (May 24-31)

Deliverables:

- Final security audit completed
- User acceptance testing with 20+ users
- Launch readiness checklist approved

Tasks:

1. Security review with IT/InfoSec team
2. Conduct UAT with diverse user group (all departments)
3. Address critical feedback and bugs
4. Prepare launch communication plan
5. Create training materials (video walkthrough, FAQ)
6. Schedule launch date and announcement

Phase 4: Advanced Features (June 1-30)

Timeline: 4 weeks (time-permitting)
Objective: Implement "nice-to-have" features if core system stable

Week 18-20: Legacy Code Ingestion (June 1-21)

Deliverables:

- One repository fully indexed and searchable
- Developers can find reusable functions via natural language

Implementation: See detailed plan in Section 3.1 above

Tasks:

1. Select target repository (high-value, frequently referenced)
2. Build function extraction script
3. Generate natural language descriptions using GPT-4
4. Ingest descriptions and code into vector DB
5. Test with developer queries
6. Document process for future repository additions

Success Metrics:

- 50+ functions indexed
- Developers successfully find relevant code in 70%+ of test queries
- Code descriptions reviewed and approved by senior developer

---

Week 21-22: Optional Enhancements (June 22-30)

Potential features (prioritize based on time available):

1. **Slack Bot:** Real-time querying within Slack
2. **Knowledge Graph:** Visual document relationships
3. **Advanced Analytics:** Query trends, popular topics, knowledge gaps
4. **Email Integration:** Ingest important email threads (Outlook/Gmail)
5. **Feedback Loop:** Users can flag incorrect answers, suggest document additions

Deliverables:

- At least one advanced feature implemented
- Comprehensive handoff documentation
- Roadmap for future development

---

4. Success Metrics & KPIs

Technical Performance Metrics

Metric	Target	Measurement Method
Query Response Time (95th percentile)	<5 seconds	Application logging
System Uptime	99%+	Monitoring dashboard
Embedding Quality (cosine similarity for relevant docs)	>0.75	Manual evaluation of 100 sample queries
Hallucination Rate	<5%	Blind testing with known-answer queries
Document Coverage	80% of high-value documents ingested	Document inventory audit

User Adoption Metrics

Metric	Target	Measurement Method
Active Users (monthly)	60% of employees	Analytics dashboard
Queries per User per Month	10+	Usage tracking
User Satisfaction Score	4/5 average	Post-query feedback
Repeat Usage Rate	70% of users return within 1 week	Behavioral analytics
Reduction in "I don't know where that document is" support tickets	40% decrease	IT ticket analysis

Business Impact Metrics

Metric	Target	Measurement Method
Time Saved per Query	10 minutes average (vs. manual search)	User surveys
Knowledge Reuse Rate	30% of queries find existing solutions/documents	Content analysis
New Employee Onboarding Time	20% reduction	HR metrics
Proposal Creation Time	15% reduction (Sales team)	Sales team survey

5. Risk Management

High-Priority Risks

Risk 1: Hallucination / Inaccurate Answers

Probability: Medium | **Impact:** High

Description: LLM generates plausible but incorrect answers, eroding user trust.

Mitigation Strategies:

- Strict system prompt: "Only answer from provided context"
- Low temperature setting (0.1) for factual accuracy
- Implement confidence scoring and display uncertainty
- "I don't know" responses when relevance is low

- Human-in-the-loop review for critical queries (HR, Finance)

Monitoring:

- User feedback ("Was this helpful?")
- Regular spot-checks by subject matter experts
- Track flagged answers for retraining

Risk 2: Data Privacy / Unauthorized Access

Probability: Medium | **Impact:** Critical

Description: User accesses documents they don't have permission to view.

Mitigation Strategies:

- RBAC implementation with thorough testing (100+ test cases)
- Audit logging of all queries and document access
- Regular permission audits by IT/HR
- Principle of least privilege (default deny, explicit allow)

Monitoring:

- Weekly audit log reviews
- Automated alerts for unusual access patterns
- Annual security penetration testing

Risk 3: Infrastructure / API Downtime

Probability: Low-Medium | **Impact:** Medium

Description: OpenAI API or Qdrant becomes unavailable, system non-functional.

Mitigation Strategies:

- Fallback LLM provider (Claude 3.5 Sonnet) configured
- Qdrant backups every 24 hours
- Application-level error handling and graceful degradation
- Status page showing system health

Monitoring:

- Automated uptime monitoring (ping every 60 seconds)

- Alert on-call person if down >5 minutes
- Monthly disaster recovery drills

Risk 4: Poor Document Quality / Outdated Information

Probability: High | **Impact:** Medium

Description: System ingests outdated or incorrect documents, provides bad advice.

Mitigation Strategies:

- Document metadata includes "last reviewed" date
- Flag documents older than 2 years in UI
- Periodic document audits by department owners
- User feedback mechanism: "This information is outdated"

Monitoring:

- Quarterly document age reports to department heads
- Track "outdated" flags from users
- Establish document review cycles

Risk 5: Scope Creep / Timeline Overrun

Probability: Medium | **Impact:** Medium

Description: Stakeholders request additional features, delaying MVP delivery.

Mitigation Strategies:

- Strict adherence to MoSCoW prioritization
- Weekly progress reviews with guidance counselor
- Clear "Won't Have" list communicated early
- Feature request backlog for post-internship

Monitoring:

- Weekly milestone tracking (Gantt chart)
- Bi-weekly stakeholder updates
- Escalation process for scope changes

Medium-Priority Risks

Risk	Mitigation
API Cost Overruns	Set monthly budget alerts, implement caching, monitor usage daily
User Adoption Resistance	Early pilot program, training sessions, champion users in each department
Technical Skill Gaps (Intern)	Regular mentorship sessions, online courses (LangChain tutorials), peer programming
Data Source Access Delays	Request permissions in Week 1, escalate blockers immediately to leadership
Document Format Incompatibilities	Build robust error handling, maintain "problematic files" queue for manual review

---

6. Resource Requirements

Human Resources

Role	Commitment	Responsibilities
Intern (You)	Full-time (40 hrs/week)	Core development, testing, documentation
Guidance Counselor	5 hrs/week	Architecture guidance, weekly reviews, stakeholder liaison
Senior Developer (Mentor)	2 hrs/week	Code reviews, technical troubleshooting
HR Representative	1 hr/week	Document classification, permission mapping
IT/Infrastructure	3 hrs total	Server setup, network configuration, monitoring
InfoSec	4 hrs total	Security review, penetration testing
Pilot Users (5 people)	2 hrs each (total 10 hrs)	User testing, feedback

Total Human Resource Cost Estimate: ~200 hours external support (non-intern)

Budget Estimate

Category	Item	Cost
Cloud Services	OpenAI API (10K queries/month)	$200-300/month

Category	Item	Cost
	Qdrant Cloud (if not self-hosted)	$100/month
	Backup Storage	$50/month
Infrastructure	VM/Server (if not existing)	$0 (assuming existing infrastructure)
Software Licenses	None (all open-source)	$0
Training	Online courses (LangChain, RAG)	$50
Miscellaneous	Buffer for unexpected costs	$100
Total (4 months)		**~$1,500**

Note: Costs significantly lower if self-hosting Qdrant and using existing company infrastructure.

---

7. Handoff & Sustainability Plan

End-of-Internship Deliverables

1. **Codebase:**
    - Production-ready application in Git repository
    - Comprehensive README with setup instructions
    - Code comments and docstrings
    - Unit tests for critical functions
2. **Documentation:**
    - User manual with screenshots
    - Administrator guide (deployment, troubleshooting)
    - API documentation (for future integrations)
    - Architecture diagram
3. **Knowledge Transfer:**
    - 2-hour recorded walkthrough of system architecture
    - Live demo to stakeholders
    - Written post-mortem: what worked, what didn't, lessons learned
    - Recommendations for future development
4. **Operational Playbooks:**
    - How to add new data sources
    - How to update embeddings model

- How to handle common errors
- How to scale system (more users, more documents)

Maintenance Plan (Post-Internship)

Recommended Ongoing Support:

- **System Administrator:** 2-4 hrs/week (monitor uptime, restart services, review logs)
- **Content Manager:** 4-6 hrs/week (review flagged documents, update classifications, respond to user feedback)
- **Developer (for enhancements):** 10 hrs/month (bug fixes, new features from backlog)

Quarterly Activities:

- Document audit and cleanup (remove outdated files)
- Permission review (update user roles as org changes)
- Performance optimization (tune search parameters)
- User satisfaction survey

Annual Activities:

- Security audit and penetration testing
- Major version upgrades (LangChain, Qdrant, LLMs)
- Evaluate new technologies (better embeddings models, graph databases)

---

8. Contingency Plans

If MVP Deadline at Risk (April 15 checkpoint)

Descope to Absolute Minimum:

1. **Keep:** Document ingestion, basic RAG, web UI
2. **Defer:** Slack integration, RBAC (use basic auth instead), metadata filtering
3. **Action:** Reassess timeline weekly, communicate openly with stakeholders

---

If Key Technology Fails

OpenAI API Unavailable:

- Switch to Claude 3.5 Sonnet (already configured as backup)

- Or use open-source model (e.g., Llama 3 via Ollama) for limited functionality

Qdrant Issues:

- Switch to Pinecone (cloud-based, pay-as-you-go)
- Or switch to Chroma (local, good for small deployments)

Streamlit Performance Issues:

- Migrate to FastAPI + React (more complex, but more scalable)
- Or use Gradio as simpler alternative

If User Adoption Low After Launch

Action Plan:

1. Conduct user interviews to understand barriers
2. Improve onboarding (video tutorials, tooltips)
3. Identify and support "champion users" in each department
4. Run weekly "office hours" for questions
5. Gamification (leaderboard for most active users)

---

9. Conclusion & Next Steps

This implementation plan provides a structured, phased approach to building IntoData's knowledge base system within the internship timeline. By prioritizing core features (MoSCoW), managing risks proactively, and maintaining realistic milestones, the project is positioned for success.

Immediate Next Steps (Week 1)

Day 1-2:

- [ ] Review and approve this implementation plan with guidance counselor
- [ ] Request API keys (OpenAI, Qdrant)
- [ ] Set up development environment

Day 3-5:

- [ ] Conduct stakeholder interviews (Sales, HR, Engineering) to refine use cases
- [ ] Begin document inventory (identify 50 high-value documents)
- [ ] Create project tracking board (Jira/Trello)

Week 2:

- [ ] Develop and test basic ingestion script
- [ ] Set up Qdrant instance
- [ ] First code commit to repository

Critical Success Factors

1. **Clear Communication:** Weekly updates to stakeholders, transparent about blockers
2. **Iterative Development:** Ship small, test frequently, gather feedback early
3. **Scope Discipline:** Resist feature creep, stick to MoSCoW priorities
4. **User-Centric:** Always test with real users, prioritize usability over complexity
5. **Documentation:** Write as you go, don't leave documentation for the end

---

This project has the potential to transform how IntoData accesses and leverages its institutional knowledge. Let's build something that makes every employee's job easier.

Document Version: 1.0
Last Updated: February 4, 2026
Author: Intern Project Lead
Reviewed By: [Guidance Counselor Name]